

Maritime CargoService: Sprint 0

Davide Chirichella, Gabriele Doti, Daniele
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026
Alma Mater Studiorum . Università di Bologna

July 04, 2026

1. Introduction

Una compagnia di trasporto marittimo di container (d'ora in poi *la committente*) intende automatizzare le operazioni di carico dei container nella hold della nave (d'ora in poi **hold**). A tal fine prevede di impiegare un robot a guida differenziale (Differential Drive Robot, d'ora in poi **cargorobot**).

L'obiettivo dello Sprint 0 è formalizzare e disambiguare i requisiti forniti dalla committente in linguaggio naturale, costruire un primo modello del sistema, evidenziare il *core business*, motivare la scelta del linguaggio di modellazione e definire un primo insieme di piani di test funzionali e il goal dello Sprint 1.

Ogni scelta è strettamente ancorata ai requisiti: il modello qui presentato serve a catturare il comportamento richiesto, senza anticipare decisioni progettuali o scelte implementative che verranno affrontate negli sprint successivi.

I requisiti del progetto sono riportati nel documento disponibile al seguente [link](#)^o

2. Requirements

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
 - lo stato attuale della hold
 - il messaggio "**Service working**" quando tutto sta procedendo correttamente
 - il messaggio "**Out of service**" se il sensore sonar misura una distanza $D > D_{FREE}$ per almeno 3 secondi (possibile guasto del sonar)

2.1. Domande aperte alla committente

Domanda al committente.

Posizione dell'IOPort nella mappa. I requisiti indicano che il cargorobot sposta

il container dall'IOPort a slot5, lasciando intendere che l'IOPort sia una cella praticabile. Come si colloca nella griglia?

Risposta della committente La collocazione dell'IOPort nella griglia è da intendersi informalmente come mostrato nella figura presente nella traccia.

Domanda al committente.

Richieste concorrenti. Il sistema deve bufferizzare più richieste di carico contemporanee, oppure una nuova richiesta ricevuta mentre il sistema è engaged viene semplicemente refusata / respinta con retrylater senza accodamento? Dai requisiti si interpreta che le richieste **non siano bufferizzate**: se il sistema è occupato, la risposta è immediata (retrylater o refused) senza code di attesa.

Risposta della committente Si conferma che le richieste non devono essere bufferizzate. Il sistema accetta le richieste mediante l'IOPort e, se questa risulta occupata, non deve essere possibile inviare altre richieste.

Domanda al committente.

Liberazione degli slot e aggiornamento della disponibilità. I requisiti descrivono il processo di carico dei container negli slot1-4, ma non specificano come venga gestita la liberazione di uno slot già occupato. Possiamo assumere che lo svuotamento degli slot sia effettuato da sistemi o operatori esterni al nostro sistema? In tal caso, con quale meccanismo viene notificato a cargoservice che uno specifico slot è stato liberato e può tornare disponibile per future prenotazioni?

Risposta della committente Non è previsto lo svuotamento degli slot. Sarà l'obiettivo di un progetto futuro.

Domanda al committente.

Ripristino dello stato Service working. I requisiti specificano che il sistema entra nello stato Out of service quando il sonar rileva una distanza $D > D_{FREE}$ per almeno 3 secondi. Non è invece specificata la condizione per il ritorno allo stato Service working. Il sistema deve tornare operativo non appena il sonar rileva $D \leq D_{FREE}$ oppure è richiesto un ulteriore intervallo di stabilità prima del ripristino del servizio?

Risposta della committente Si conferma che l'interpretazione è corretta.

Domanda al committente.

Indicazioni sulla realizzazione dell'IOPort. Ci sono indicazioni sulla realizzazione dell'IOPort?

Risposta della committente Si conferma che bisogna svilupparla come una web GUI.

3. Requirement analysis

3.1. Motivazione dell'uso del linguaggio QAK

QAK è il linguaggio messo a disposizione dalla nostra software house per modellare sistemi software distribuiti, particolarmente espressivo nel formalizzare il concetto di **attore autonomo** e di **messaggio**, riducendo di molto l'“Abstraction Gap” tra requisiti nel contesto di un sistema distribuito eterogeneo.

I requisiti descrivono un sistema composto da entità che ricevono richieste, inviano risposte, osservano eventi e coordinano dispositivi. Java, preso come linguaggio general purpose, esprime invece in modo naturale soprattutto interazioni tramite chiamate di procedura, spesso sincrone e bloccanti. Questo rischia di introdurre precocemente dettagli implementativi che distolgono dal problema principale: formalizzare il comportamento osservabile richiesto dalla committente.

QAK introduce invece come concetti primitivi quelli di **attore**, **messaggio**, **request**, **reply**, **dispatch** ed **event**, rendendo il modello più vicino al dominio del problema. La natura **reattiva** e **proattiva** di servizi che devono rispondere a stimoli esterni e avviare autonomamente sequenze di azioni è infatti catturata in modo naturale da un attore QAK, cosa che un POJO, componente passivo attivato da chiamate sincrone, non catturerebbe altrettanto bene.

Dal modello QAK viene inoltre generato automaticamente codice Kotlin eseguibile, il che consente di disporre di un **primo prototipo osservabile già nello Sprint 0**, prima ancora di scrivere una riga di logica applicativa.

3.2. Vocabolario

TERMINE	SIGNIFICATO
engaged	Stato nel quale il sistema sta gestendo una richiesta di carico.
disengaged	Stato nel quale il sistema può accettare una nuova richiesta.
Out of service	Stato nel quale il servizio non può accettare richieste a causa del malfunzionamento del sonar.
hold	Modello logico della stiva e dell'occupazione degli slot.
slot libero	Primo slot disponibile tra slot1-slot4.

3.3. Contesti logici

Dai requisiti emerge che il sistema non è naturalmente concentrato in un unico processo; le entità sono quindi state distribuite su context distinti, ciascuno dei quali rappresenta un nodo di elaborazione.

CONTESTO	COMPONENTI E RESPONSABILITÀ
----------	-----------------------------

ctxCargoService	È il nucleo del comportamento richiesto e punto di orchestrazione del ciclo di carico. Contiene il cargoservice.
ctxIOPort	Raggruppa le entità dedicate all'interazione con l'utente. Contiene l'IOPort (con display e pushbutton)
ctxDevices	Raggruppa i dispositivi presenti nella hold: sonar, LED, hold e markerdevice.
ctxRobot	Raggruppa le entità legate al cargorobot e alla movimentazione richiesta.

3.4. Formalizzazione dei messaggi QAK

Dai requisiti, l'unica richiesta che si evince è quella di carico, che viene modellata come request in qak. La richiesta viene inviata dal customer al cargoservice tramite l'IOPort.

```
Request load_request      : loadRequest(none)
Reply load_accepted      : loadAccepted(slotID) for load_request // accettazione
Reply load_retrylater    : loadRetryLater(none) for load_request // rinvio
                           temporaneo
Reply load_refused       : loadRefused(none) for load_request // rifiuto definitivo
```

3.5. Macro-componenti e natura software

3.5.1. cargoservice

Il cargoservice è l'**orchestratore** principale del sistema. Gestisce le richieste di carico, verifica le condizioni della hold, controlla i timeout e coordina le altre componenti.

Il componente è da sviluppare.

```
Request load_request      : loadRequest(none)
Reply load_accepted      : loadAccepted(slotID) for load_request
Reply load_retrylater    : loadRetryLater(none) for load_request
Reply load_refused       : loadRefused(none) for load_request

Context ctxcargoservice ip [host="localhost" port=8050]

QActor cargoservice context ctxcargoservice {

  State s0 initial {
    println("cargoservice | started")
  }
  Goto disengaged

  State disengaged {
    println("cargoservice | DISENGAGED: waiting for load_request...")
  }
  Transition t0
    whenRequest load_request → handle_load_request

  State handle_load_request {
```

```

        println("cargoservice | MOCK: handling load_request with accepted")

        replyTo load_request with load_accepted : loadAccepted(slot1)
        // replyTo load_request with load_retrylater : loadRetryLater(none)
        // replyTo load_request with load_refused : loadRefused(none)
    }
    Goto engaged

    State engaged {
        println("cargoservice | ENGAGED: slot reserved")
    }
}

```

Il codice è disponibile al seguente [link](#)^o

3.5.2. cargorobot

Il **cargorobot** è il sottosistema responsabile della movimentazione fisica del container all'interno della hold.

Dopo una discussione con la committente, si conviene che la responsabilità di decidere la sequenza applicativa resta in capo al **cargoservice**, il quale movimenterà il cargorobot.

Sarà successivamente opportuno analizzare quali software già disponibili nella software house o in rete siano adeguati al problema. In particolare, andrà valutato se riutilizzare software come **robosmart26** o **robotservice26**.

Di seguito viene riportata una formalizzazione del suo comportamento, che si limita a ricevere comandi di spostamento e a eseguirli. La decisione sulla loro realizzazione concreta è rinviata all'analisi del problema.

```

Context ctxrobot      ip [host="localhost" port=8053]

QActor cargorobot context ctxrobot {

    State s0 initial {}
    Goto work

    State work {
        println("cargorobot | WORK: move container from IOPort to slot5 and then
to reserved slot")
    }
}

```

Il codice è disponibile al seguente [link](#)^o

3.5.3. IOPort

L'IOPort rappresenta l'interfaccia tra customer e sistema. Dopo una discussione con la committente, si comprende che esso viene interpretato come una Web GUI composta

da un pushbutton e da un display. Dai requisiti si deduce che è IOPort ad emettere la richiesta **load_request** verso **cargoservice** e mostrare informazioni di stato.

Nel modello QAK può essere rappresentato come attore per modellarne il comportamento comunicativo, non perché la sua implementazione finale debba necessariamente essere QAK.

```
Request load_request      : loadRequest(none)

Reply load_accepted       : loadAccepted(SLOTID) for load_request
Reply load_retrylater     : loadRetryLater(none) for load_request
Reply load_refused        : loadRefused(none) for load_request

Context ctxioport         ip [host="localhost" port=8050]

QActor ioport context ctxioport {

    State s0 initial {}
    Goto work

    State work {
        println("ioport | PUSHBUTTON: customer pressed pushbutton")
        request cargoservice -m load_request : loadRequest(none)
    }
    Transition t0
        whenReply load_accepted    → accepted
        whenReply load_retrylater  → retrylater
        whenReply load_refused     → refused

    State accepted {
        println("ioport | DISPLAY: load_accepted received") color green
        // il display mostra lo slot riservato
    }
    Goto work

    State retrylater {
        println("ioport | DISPLAY: load_retrylater received") color yellow
        // il display mostra richiesta rinviata
    }
    Goto work

    State refused {
        println("ioport | DISPLAY: load_refused received") color red
        // il display mostra richiesta rifiutata
    }
    Goto work
}
```

Il codice è disponibile al seguente [link](#)^o

3.5.4. sonar

Il sonar rileva la presenza di un container.

Il dispositivo fisico è considerato fornito ed è collegato al PicoW, mentre il software di integrazione è da sviluppare.

La formalizzazione del sonar come attore è disponibile al seguente [link](#)^o

```
QActor sonar context ctxdevices {  
  
    State s0 initial {}  
    Goto work  
  
    State work{  
        println("sonar | WORK: misuring...") color blue  
    }  
}
```

3.5.5. markerdevice

Il markerdevice etichetta i container depositati nello slot5 e notifica il completamento della marcatura.

Il dispositivo è considerato disponibile in forma simulata, mentre il software di controllo è da sviluppare.

La formalizzazione del markerdevice come attore è disponibile al seguente [link](#)^o

```
QActor markerdevice context ctxdevices {  
  
    State s0 initial {}  
    Goto work  
  
    State work{  
        println("markerdevice | WORK: marking") color yellow  
    }  
}
```

3.5.6. LED

Il LED è un dispositivo fisico usato per rendere osservabile lo stato **engaged** del sistema.

La frase dei requisiti “*while engaged, the system blink a Led*” viene formalizzata nel seguente modo: quando il cargoservice entra nello stato **engaged**, il LED deve lampeggiare; quando il sistema torna nello stato **disengaged**, il LED deve essere spento.

Dopo una consultazione con la committente, si è definito che il LED è considerato un dispositivo fisico integrato nel PicoW che gestisce anche il sonar. Il software di controllo del LED è quindi da sviluppare o integrare nel software eseguito sul PicoW.

La formalizzazione del LED come attore è disponibile al seguente [link](#)^o

```
QActor led context ctxdevices {  
  
    State s0 initial {}  
  
}
```



```

Goto work

State work{
    println("led | WORK: led off or led on") color green
}
}

```

3.5.7. hold

La hold è l'entità che rappresenta logicamente la stiva e lo stato di occupazione degli slot.

Il componente è da sviluppare.

Può essere formalizzata come una struttura dati composta da Celle, ovvero una matrice bidimensionale. Ogni Cella può indicare uno spazio libero, un ostacolo, la HOME, il SONAR, l'IOPORT o uno slot (**slot1–slot4** e lo **slot5** usato per la marcatura).

```

public enum CellType {
    FREE, OBSTACLE, HOME, SONAR, IOPORT, SLOT1, SLOT2, SLOT3, SLOT4, SLOT5
}

public class Hold {

    private final CellType[][] cells = {

        { FREE,    HOME,    FREE,    FREE,    FREE,    FREE,    FREE },
        { SONAR,   FREE,    SLOT1,   OBSTACLE, SLOT2,   FREE,    FREE },
        { FREE,    FREE,    FREE,    FREE,    FREE,    SLOT5,   FREE },
        { FREE,    FREE,    SLOT3,   OBSTACLE, SLOT4,   FREE,    FREE },
        { FREE,    FREE,    FREE,    FREE,    FREE,    FREE,    FREE },
        { FREE,    FREE,    FREE,    FREE,    FREE,    FREE,    FREE },
        { IOPORT,  FREE,    FREE,    FREE,    FREE,    FREE,    FREE }
    };

    // slot occupati (SLOT1 ... SLOT5)
    private final boolean[] occupiedSlots = new boolean[5];
}

```

4. Test plan

I requisiti dello Sprint 0 descrivono esclusivamente il comportamento osservabile del cargoservice in risposta a una richiesta di carico, senza fornire dettagli implementativi sui collaboratori o sulla logica interna del sistema.

Di conseguenza, il piano di test si limita a **verificare i casi funzionali** direttamente deducibili dai requisiti e formalizzati nel modello sviluppato.

Il codice dei test è disponibile al seguente [link](#)°

```

package test

import org.junit.Assert.assertTrue

```

```

import org.junit.Assert.fail
import org.junit.After
import org.junit.Before
import org.junit.Test
import unibo.basicomm23.interfaces.Interaction
import unibo.basicomm23.tcp.TcpClientSupport
import unibo.basicomm23.utils.CommUtils

class TestPlanSprint0 {
    private var conn: Interaction? = null

    @Before
    fun setup() {
        CommUtils.outcyan("Connessione al cargoservice (porta 8050)")
        try {
            conn = TcpClientSupport.connect("127.0.0.1", 8050, 10)
        } catch (e: Exception) {
            fail("Errore di connessione TCP: assicurati che il server
cargoservice sia avviato.")
        }
    }

    @After
    fun teardown() {
        conn?.close()
        CommUtils.outcyan("Connessione chiusa")
    }

    @Test
    fun testLoadRequest() {
        CommUtils.outmagenta("Invio load_request e verifica risposta QAK")
        try {

            // firma della richiesta in base al modello definito
            val requestMsg = "msg(load_request, request, testunit,
cargoservice, loadRequest(none), 1)"

            // invio richiesta
            val reply = conn?.request(requestMsg)
            CommUtils.outgreen("Risposta ricevuta dal server: $reply")

            // la risposta deve essere una delle 3 reply previste dal modello
            val isValidReply = reply != null && (
                reply.contains("load_accepted") ||
                reply.contains("load_retrylater") ||
                reply.contains("load_refused")
            )
            assertTrue("Il server non ha restituito una risposta di carico
valida!", isValidReply)

        } catch (e: Exception) {
            fail("Test fallito durante l'esecuzione: ${e.message}")
        }
    }
}

```

5. Architettura

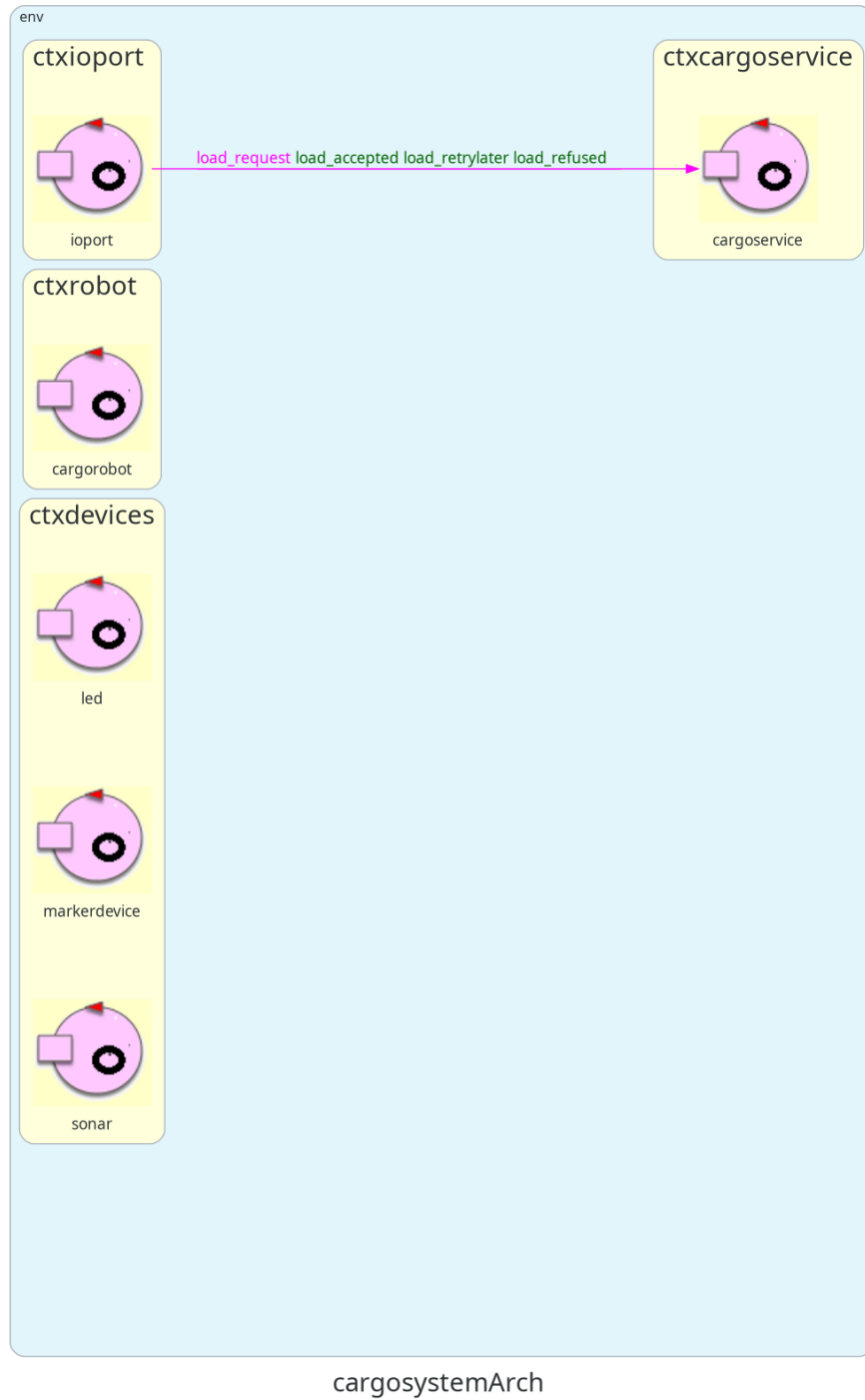


Figure 1: Architettura definita nello Sprint0.

6. Goal del successivo Sprint 1 (DA AGGIORNARE)

Goal: realizzare un primo prototipo eseguibile del **cargoservice** che implementi il comportamento principale descritto dai requisiti mediante collaboratori simulati.

Al termine dello Sprint1 il sistema dovrà essere in grado di:

- ricevere una **load_request** proveniente dall'IOPort;
- verificare lo stato della hold, dell'IOPort e del servizio;
- produrre una delle tre risposte previste:
 - **load_accepted(slotID)**;
 - **load_retrylater**;
 - **load_refused**;
- tenere traccia dello stato della hold e delle prenotazioni degli slot;
- gestire gli stati **engaged** e **disengaged**;
- pilotare il LED in funzione dello stato del sistema;

tramite collaboratori simulati.

7. Team di lavoro

NOME E COGNOME	MATRICOLA
Davide Chirichella	0001222371
Gabriele Doti	0001245897
Daniele Maccagnan	0001247340